

Data Lab 实验报告

张心淳

2025201884

0. 提交说明

- 源文件: `bits.c`
- 报告源文件: `report.md`
- 报告 PDF: `report.pdf`
- 图片目录: `imgs/` (本报告未使用图片)

提交目录结构:

```
report/  
  report.md  
  report.pdf  
  bits.c  
  imgs/
```

1. 实验目标

本实验要求在限定运算符和最大操作数的约束下, 实现若干位运算与浮点数转换函数, 加深对补码、位级运算、逻辑移位、IEEE 754 单精度/双精度表示以及舍入规则的理解。最终使用 `./btest` 和 `test.py` 验证正确性。

2. 实现

具体实现见 `bits.c`, 报告不重复粘贴完整代码。

3. 位运算函数说明

3.1 `bitAnd`

利用德摩根律:

$$x \& y = \sim(\sim x \mid \sim y)$$

因此只需对 `x`、`y` 分别取反, 做按位或, 再整体取反即可。

3.2 `bitXor`

异或表示两位不同。实现中先排除“同为 1”的情况, 再排除“同为 0”的情况, 两者取交集:

$$x \wedge y = \sim(x \& y) \& \sim(\sim x \& \sim y)$$

3.3 samesign

题目规定 0 既不是正数也不是负数，因此需要特殊处理。

实现分两部分：

1. 判断 `x`、`y` 是否同时为 0 或同时为非 0；
2. 判断 `x`、`y` 的符号位是否相同。

两部分同时成立时返回 1，否则返回 0。这样 `samesign(0, 0) = 1`，而 `samesign(0, 1) = 0`。

3.4 logtwo

对于正数 `v`，其以 2 为底的对数等于最高有效位的位置。

采用二分思想，依次判断右移 16、8、4、2、1 位后是否仍大于 0，从而确定最高位落在哪一段。每一段若成立，就把对应位移量累加到结果中。最后返回累加值。

3.5 byteSwap

要交换第 `n` 和第 `m` 个字节，先分别取出这两个字节，异或得到它们的差异。然后把差异分别异或回原来的两个字节位置。因为异或两次会抵消原值并写入另一个字节，所以可以实现交换。

字节偏移通过 `n << 3` 和 `m << 3` 计算。

3.6 reverse

反转 32 位位序时，先保存原最高位。循环中不断取当前最低位，并拼接到结果中，同时原数右移。循环结束后处理之前保存的最高位，将其放到最低位。

该方法避免直接写 32 次重复操作，同时满足运算符限制。

3.7 logicalShift

C 语言中有符号整数右移通常是算术右移，会在高位补符号位。实现中先得到算术右移结果，再构造一个掩码，把多补进来的符号位异或掉，从而得到逻辑右移效果。

3.8 leftBitCount

若 `x` 最高位为 0，则左侧连续 1 的个数为 0。

若最高位为 1，则对 `~x` 查找最高位 1 的位置，该位置即原数左侧连续 1 的结束位置。使用与 `logtwo` 类似的二分法，依次检查高 16、8、4、2、1 位是否包含 1，并累加位置。全 1 时返回 32。

4. 浮点函数说明

4.1 float_i2f

将 `int` 转换为单精度浮点位表示。

主要步骤：

1. 提取符号位；

2. 对负数取绝对值;
3. 若为 0, 直接返回 0;
4. 找到最高有效位位置 e , 得到阶码 $e + 127$;
5. 根据 e 与 23 的关系生成尾数:
 - 若 $e \leq 23$, 尾数左移;
 - 若 $e > 23$, 尾数右移, 并进行就近舍入到偶数;
6. 若尾数进位导致溢出, 则阶码加一, 尾数清零;
7. 拼接符号、阶码和尾数。

4.2 floatScale2

计算 $2 * f$ 的位表示。

分类处理:

- 若阶码全 1, 说明是 NaN 或 Inf, 直接返回原值;
- 若阶码为 0, 说明是非规格化数, 左移一位实现乘 2;
- 若阶码为 0xfe, 乘 2 后会变成 Inf, 返回对应 Inf;
- 否则阶码加一。

4.3 float64_f2i

将 64 位 IEEE 754 浮点数转换为 32 位有符号整数, 向零舍入。

主要步骤:

1. 从高 32 位提取符号、11 位阶码, 从高低位共同提取尾数;
2. 若阶码全 1, 返回溢出标志;
3. 若阶码为 0 或实际指数小于 0, 返回 0;
4. 若实际指数大于 30, 返回溢出;
5. 根据实际指数与 20 的关系, 拼接整数部分;
6. 根据符号处理负数, 并检查正负溢出。

4.4 floatPower2

计算 2.0^x 的单精度浮点表示。

按范围分段:

- $x > 127$: 结果过大, 返回 +INF;
- $x < -149$: 结果过小, 返回 0;
- $-149 \leq x < -126$: 非规格化数, 返回 $1 \ll (x + 149)$;
- 否则为规格化数, 阶码为 $x + 127$, 返回 $(x + 127) \ll 23$ 。

5. 验证结果

- 所有函数均未使用题目禁止的运算符，并未超过对应 `Max ops`。
- 使用 `./btest` 对各函数进行验证，结果均正确。
- 调试完成后移除了 `printf`，运行 `test.py` 完整测试通过。
- 报告中未粘贴大段代码，完整实现见 `bits.c`。

6. 总结

本实验主要训练了位级等价变换、二分查找最高有效位、逻辑右移中符号位处理、异或交换字节以及 IEEE 754 浮点数的表示和舍入规则。

实现过程中最关键的是：

1. 用德摩根律和异或性质减少运算符使用；
2. 用二分法高效定位最高有效位；
3. 正确处理逻辑右移补位；
4. 在浮点数转换中严格处理规格化、非规格化、舍入和溢出情况。

通过本实验，对整数补码和浮点编码有了更具体的理解。

